

- Description

This module manages a smart farm irrigation system, ensuring that:

1. **Soil moisture levels are continuously monitored to maintain optimal crop growth conditions.**
2. **If soil moisture drops below a predefined threshold, the irrigation system is activated.**
3. **If excessive moisture is detected, irrigation is paused to prevent overwatering.**
4. **A log of all moisture readings, irrigation actions, and system optimizations is maintained for agricultural efficiency.**

This system helps conserve water, improve crop yield, and optimize irrigation scheduling.

- Specification

module SmartFarmIrrigationControl

var currentMoisture, maxMoisture, minMoisture: real

var irrigationActive: boolean

var log: sequence of string

process updateMoisture(changeAmount: real)

functional scenario form:

currentMoisture $\geq$ minMoisture && currentMoisture = currentMoisture + changeAmount, Soil moisture updated successfully.

||

currentMoisture < minMoisture && irrigationActive = true, log = log $\oplus$ ("Dry Soil Alert: " + currentMoisture + "%, Activating Irrigation."), Dry soil detected, irrigation activated.

||

currentMoisture > maxMoisture && irrigationActive = false, log = log $\oplus$ ("Excess Moisture Alert: " + currentMoisture + "%, Pausing Irrigation."), Excess moisture detected, irrigation paused.

end process

end module

- Java Code

```
package code;
import java.util.*;
class SmartIrrigation {
    private double maxMoisture;
    private double minMoisture;
    private double currentMoisture;
    private boolean irrigationActive;
    private List<String> log;
    public SmartIrrigation(double maxMoisture, double minMoisture, double initialMoisture) {
        this.maxMoisture = maxMoisture;
```

```

        this.minMoisture = minMoisture;
        this.currentMoisture = initialMoisture;
        this.irrigationActive = false;
        this.log = new ArrayList<>();
    }

    public void updateMoisture(double changeAmount) {
        currentMoisture += changeAmount;
        manageIrrigation();
        String message = String.format("Current Soil Moisture: %.2f%%",
currentMoisture);
        if (currentMoisture < minMoisture) {
            message += " | ALERT: Dry Soil! Activating Irrigation.";
        } else if (currentMoisture > maxMoisture) {
            message += " | INFO: Excess Moisture! Pausing Irrigation.";
        } else {
            message += " | Moisture Level is Optimal.";
        }
        System.out.println(message);
        log.add(message);
    }

    private void manageIrrigation() {
        if (currentMoisture < minMoisture) {
            irrigationActive = true;
        } else if (currentMoisture > maxMoisture) {
            irrigationActive = false;
        }
    }

    public void printLog() {
        System.out.println("\nIrrigation System Log:");
        for (String logEntry : log) {
            System.out.println(logEntry);
        }
    }
}

public class c56_SmartFarmSystem {
    public static void main(String[] args) {
        SmartIrrigation irrigation = new SmartIrrigation(60.0, 30.0, 45
.0);

        // testcase-VT:
        //     irrigation.updateMoisture(-20.0);
        //     irrigation.updateMoisture(15.0);
        //     irrigation.updateMoisture(25.0);
        //     irrigation.updateMoisture(-10.0);
        //     irrigation.printLog();
    }
}

```

```

        // testcase-FT:
//      irrigation.updateMoisture(0.0);
//      irrigation.updateMoisture(-1.7976931348623157E308);
//      irrigation.updateMoisture(1.7976931348623157E308);
//      irrigation.updateMoisture(-8.812221795584368E307);
//      irrigation.printLog();

        // testcase-Z3:
//      irrigation.updateMoisture(-20);
//      irrigation.updateMoisture(15);
//      irrigation.updateMoisture(25);
//      irrigation.updateMoisture(-10);
//      irrigation.printLog();

        // testcase-UVT:
        irrigation.updateMoisture(-20.0);
        irrigation.updateMoisture(15.0);
        irrigation.updateMoisture(25.0);
        irrigation.updateMoisture(-10.0);
        irrigation.printLog();
    }
}

```

● Coverage report

1. Test case number: 4
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.2 %	10	832	842
✓ src	1.2 %	10	832	842
✓ code	1.2 %	10	832	842
✓ c56_SmartFarmSystem.java	100.0 %	10	0	10
> c56_SmartFarmSystem		0	0	0
> SmartIrrigation	100.0 %	10	0	10

3. Vibration testing instruction coverage: 97.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.7 %	132	20,148	20,280
✓ src	0.7 %	132	20,148	20,280
✓ code	0.7 %	132	20,148	20,280
✓ c56_SmartFarmSystem.java	97.8 %	132	3	135
> c56_SmartFarmSystem	88.0 %	22	3	25
> SmartIrrigation	100.0 %	110	0	110

4. Fuzz testing branch coverage: 80%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.0 %	8	834	842
✓ src	1.0 %	8	834	842
✓ code	1.0 %	8	834	842
✓ c56_SmartFarmSystem.java	80.0 %	8	2	10
> c56_SmartFarmSystem		0	0	0
> SmartIrrigation	80.0 %	8	2	10

5. Fuzz testing instruction coverage: 92.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.6 %	125	20,155	20,280
▼ src	0.6 %	125	20,155	20,280
▼ code	0.6 %	125	20,155	20,280
▼ c56_SmartFarmSystem.java	92.6 %	125	10	135
> c56_SmartFarmSystem	88.0 %	22	3	25
> SmartIrrigation	93.6 %	103	7	110

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.2 %	10	832	842
▼ src	1.2 %	10	832	842
▼ code	1.2 %	10	832	842
▼ c56_SmartFarmSystem.java	100.0 %	10	0	10
> c56_SmartFarmSystem		0	0	0
> SmartIrrigation	100.0 %	10	0	10

7. Z3 instruction coverage: 97.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.7 %	132	20,148	20,280
▼ src	0.7 %	132	20,148	20,280
▼ code	0.7 %	132	20,148	20,280
▼ c56_SmartFarmSystem.java	97.8 %	132	3	135
> c56_SmartFarmSystem	88.0 %	22	3	25
> SmartIrrigation	100.0 %	110	0	110

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.2 %	10	832	842
▼ src	1.2 %	10	832	842
▼ code	1.2 %	10	832	842
▼ c56_SmartFarmSystem.java	100.0 %	10	0	10
> c56_SmartFarmSystem		0	0	0
> SmartIrrigation	100.0 %	10	0	10

9. Unimproved V-method instruction coverage: 97.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.7 %	132	20,148	20,280
▼ src	0.7 %	132	20,148	20,280
▼ code	0.7 %	132	20,148	20,280
▼ c56_SmartFarmSystem.java	97.8 %	132	3	135
> c56_SmartFarmSystem	88.0 %	22	3	25
> SmartIrrigation	100.0 %	110	0	110